Memory-Bound by Design: An HPCG-Inspired Evaluation of Shared- and Distributed-Memory Parallel Conjugate Gradient Solvers

[Author Name 1], [Author Name 2], [Author Name 3], [Author Name 4]

Department of Computer Science and Engineering, [College Name] — BE CSE High Performance Computing Lab, 2026

***Abstract*—The HPL benchmark ranks supercomputers by dense floating-point throughput, yet most scientific applications are limited by memory bandwidth and communication. The HPCG benchmark was proposed to capture that behaviour. We present an HPCG-inspired, fully tested Conjugate Gradient (CG) benchmark for the 3D Poisson problem in CSR format with sequential, OpenMP, MPI (replicated-vector and halo-exchange communication) and hybrid MPI+OpenMP implementations. On a 2.1-million-unknown problem every implementation converges in exactly 296 iterations to a relative residual below 10^{-8}. The best speedup is 1.40× on 8 cores, which matches the measured 1.32× growth of sustainable memory bandwidth; a roofline analysis places all implementations at ≈90% of the bandwidth bound. At 8 processes halo exchange is 1.69× faster than replicated-vector communication, which spends 51% of its time communicating. The results reproduce, at laptop scale, the central argument for HPCG: sparse iterative solvers are bound by bytes per second, not flops per second.**

***Index Terms*—Conjugate Gradient, HPCG, sparse matrix–vector product, OpenMP, MPI, roofline model, memory bandwidth.**

I. Introduction

For three decades the TOP500 list has ranked systems with High-Performance LINPACK (HPL), whose LU factorisation is dominated by dense matrix–matrix multiplication and therefore by peak arithmetic rate [7]. Dongarra and Heroux observed that this metric increasingly diverges from the performance of real applications, which are dominated by sparse operations, irregular memory access and global reductions, and proposed the High Performance Conjugate Gradient (HPCG) benchmark [1]–[3].

This work rebuilds the computational core of HPCG at educational scale and uses it to answer one question experimentally: *how do shared-memory and distributed-memory parallelisation techniques perform on a CG workload, and why?* Our contributions are: (i) four verified CG implementations sharing identical kernels and a common flop model; (ii) a same-problem comparison of two MPI communication schemes; (iii) a quantitative explanation of the measured scalability with a measured-bandwidth roofline model. The software is not the official HPCG and no ranking claim is made.

II. Mathematical Formulation

A. Model problem

We discretise $-\nabla^2 u = f$ on the unit cube with homogeneous Dirichlet boundaries using the 7-point finite-difference stencil on an $n_x \times n_y \times n_z$ grid (lexicographic ordering), giving the sparse system $Ax = b$ with

$$A_{ii} = 6, \quad A_{ij} = -1 \ \text{ if } j \in N(i), \quad A_{ij} = 0 \ \text{otherwise}, \tag{1}$$

where N(i) are the in-grid face neighbours. Thus $\text{nnz} = n + 2[(n_x-1)n_y n_z + n_x(n_y-1)n_z + n_x n_y(n_z-1)]$, i.e. ≤7 non-zeros per row (>99.99% sparsity at 128^3). The right-hand side is $b = A\cdot 1$, so the exact solution $x^* = 1$ is known, as in HPCG.

Positive definiteness. A is symmetric, has positive diagonal, is weakly diagonally dominant with strict dominance on boundary rows, and is irreducible (the grid graph is connected); an irreducibly diagonally dominant symmetric matrix with positive diagonal is SPD. Moreover A is the Kronecker sum of three 1-D $[-1\ 2\ -1]$ matrices, so

$$\lambda_k = \Sigma_{d\in\{x,y,z\}} \left(2 - 2\cos\frac{k_d\pi}{n_d+1}\right), \quad \kappa(A) = O(h^{-2}). \tag{2}$$

Our test suite verifies (2) numerically: dense Cholesky succeeds for $A - (\lambda_{\min} - 10^{-6})I$ and fails for $A - (\lambda_{\min} + 10^{-6})I$.

B. Conjugate Gradient algorithm

CG [4] minimises the A-norm error over the Krylov space $K_k(A, r_0)$ and satisfies [5]

$$\|e_k\|_A \le 2\left(\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}\right)^k \|e_0\|_A, \tag{3}$$

so the iteration count grows like $\sqrt{\kappa} = O(h^{-1})$, i.e. linearly with the grid dimension. Algorithm 1 is used by all four implementations; the two dot products are fused with the preceding sweep to save a pass over memory.

Algorithm 1 Unpreconditioned CG (fused kernels)

1. $r \leftarrow b - Ax_0$; $p \leftarrow r$; $\rho \leftarrow r^T r$; $\beta_n \leftarrow \|b\|$
2. **while** $\sqrt{\rho} > \text{tol}\cdot\beta_n$ **and** $k < k_{\max}$ **do**
3. $q \leftarrow Ap$; $\sigma \leftarrow p^T q$ *(SpMV + dot, one sweep)*
4. **if** $\sigma \le 0$ **then** stop (breakdown)
5. $\alpha \leftarrow \rho/\sigma$; $x \leftarrow x + \alpha p$; $r \leftarrow r - \alpha q$; $\rho' \leftarrow r^T r$ *(one sweep)*
6. $\beta \leftarrow \rho'/\rho$; $\rho \leftarrow \rho'$; $p \leftarrow r + \beta p$; $k \leftarrow k+1$
7. **end while**

C. Work, traffic and arithmetic intensity

Per iteration CG performs one SpMV (2·nnz flops), two dot products and three AXPY-type updates (2n flops each):

$$F_{it} = 2\cdot nnz + 10n, \quad Q_{it} \approx 12\cdot nnz + 96n \text{ bytes}, \tag{4}$$

where Q counts 8-byte values and 4-byte column indices (12 B per non-zero) plus streamed vector traffic. With nnz/n ≈ 6.95 the arithmetic intensity is

$$I = \frac{F_{it}}{Q_{it}} \approx 0.133 \text{ flop/byte}, \quad P \le \min(P_{peak}, I\cdot B), \tag{5}$$

the roofline bound [8] for sustained memory bandwidth B. Speedup and efficiency on p = processes × threads units are

$$S(p) = \frac{T_{seq}}{T(p)}, \quad E(p) = \frac{S(p)}{p} \times 100\%. \tag{6}$$

III. Implementation

Sequential is the baseline. *OpenMP* turns each fused sweep into an `omp parallel for schedule(static)` loop with `reduction(+)` for the dot products, avoiding data races; the implicit barrier is the only synchronisation. *MPI* assigns contiguous row blocks (sizes differ by ≤1 for uneven splits); each rank generates only its own rows. The entries of p required by local rows form a contiguous window refreshed before every SpMV by either (a) **Allgatherv**, which replicates the whole vector (O(n) traffic per rank), or (b) **halo exchange** via `MPI_Isend/Irecv` of at most one $n_x n_y$ grid plane per neighbour. Dot products use `MPI_Allreduce`; all branching uses reduced values, so every rank issues identical collective sequences. *Hybrid* combines both under `MPI_THREAD_FUNNELED`, calling MPI only from the master thread between parallel regions (the "master-only" style [11]).

IV. Experimental Setup

Experiments ran on an Intel Core 7 240H laptop (16 logical CPUs) inside an Ubuntu 24.04 container (Docker/WSL2, ≈11.5 GiB visible), compiled with g++ 13.3 `-O3`, OpenMP 4.5 and Open MPI 4.1.6 (`--bind-to none`). The strong-scaling problem is the 128^3 grid (n = 2,097,152; nnz = 14,581,760; ≈263 MiB), tol = 10^{-8}. Only the solve is timed (generation and allocation excluded); MPI time is the maximum over ranks. Each configuration runs one warm-up and five timed solves; medians are reported. Bandwidth was measured with a STREAM-style triad [9]. In total 50 configurations were measured (0 failed, 0 skipped).

V. Results and Analysis

A. Correctness

All ten CTest targets pass, including MPI tests on 1–4 ranks (3 ranks exercises uneven distribution). In the strong-scaling study every implementation required exactly **296 iterations** and reached true relative residual 9.79×10^{-9} with $\max|x-1| = 7.69\times10^{-8}$, so all timing comparisons are like-for-like.

Table I
Strong scaling, 128^3 grid (median of 5 runs)

Configuration	p	T (s)	S	E (%)	GFLOPS	Comm.
Sequential	1	5.953	1.00	100	2.50	–
OpenMP	2	4.540	1.31	65.6	3.28	–
OpenMP	4	4.455	1.34	33.4	3.34	–
OpenMP	**8**	**4.240**	**1.40**	**17.5**	**3.51**	**–**
OpenMP	16	6.634	0.90	5.6	2.24	–
MPI Allgatherv	2	5.148	1.16	57.8	2.89	13.5%
MPI Allgatherv	8	7.446	0.80	10.0	2.00	50.6%
MPI halo	2	4.730	1.26	62.9	3.15	3.0%
MPI halo	8	4.405	1.35	16.9	3.38	11.5%
Hybrid 4×2	8	4.269	1.39	17.4	3.49	5.1%
Hybrid 4×4	16	10.229	0.58	3.6	1.45	45.9%

B. Strong scaling

Table I and Fig. 1 show that every implementation saturates near S ≈ 1.4: the best result is OpenMP with 8 threads (5.95 s → 4.24 s). Efficiency falls from 65.6% at p = 2 to 17.5% at p = 8. This is not an Amdahl serial-fraction effect — virtually all work is parallel — but a bandwidth effect, quantified in Section V-D. Using all 16 logical CPUs is slower than 8 for every model: hyper-threads add no memory bandwidth while increasing contention.

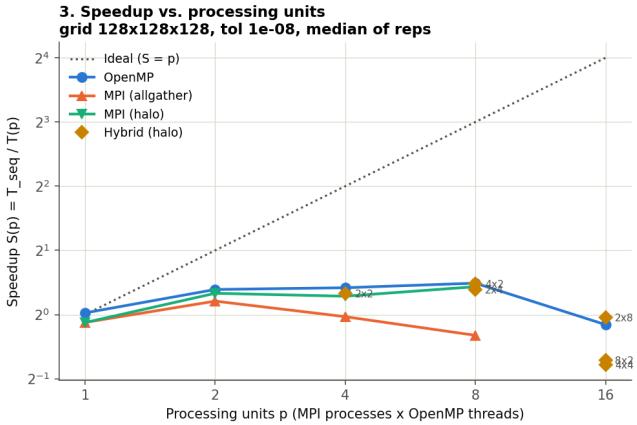


Fig. 1. Speedup versus processing units ($\log_2$ axes) for the 128^3 problem. Dotted line: ideal S = p. All implementations flatten at ≈1.4.

C. Communication overhead

Fig. 2 separates computation from communication. With Allgatherv each rank receives n − n/p values per iteration (1.84 M doubles ≈ 14.7 MB at p = 8); its communication share grows from 13.5% to 50.6% and at p = 8 it is *slower than sequential* (S = 0.80). Halo exchange receives at most 2·16,384 values per rank (≈56× less), keeps the share at ≤11.5% and is **1.69× faster** at p = 8. The hybrid 4×2 layout lowers it further to 5.1% by using fewer ranks.

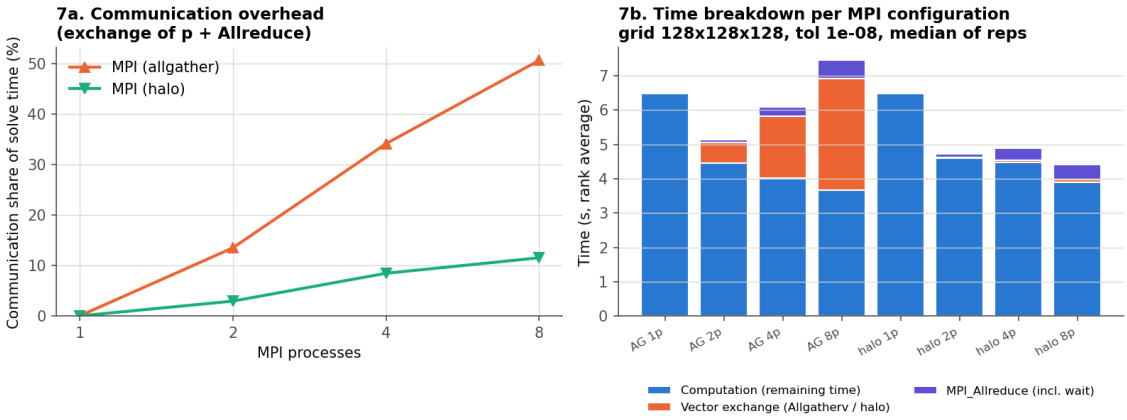


FIG. 2. (a) Communication share of solve time versus MPI processes for both exchange schemes; (b) per-configuration time breakdown into computation, vector exchange and `MPI_Allreduce` (rank-averaged, 128^3 grid).

D. Roofline and bandwidth analysis

The STREAM-style triad sustains 21.9 GB/s with one thread and at most 28.9 GB/s with eight — a gain of only **1.32**×, which matches the observed speedup ceiling. Inserting I = 0.133 into (5) gives attainable performance of 2.9 GFLOPS (1 thread) and 3.8 GFLOPS (chip). The measured 2.50 and 3.51 GFLOPS correspond to effective bandwidths of 18.8 and 26.3 GB/s, i.e. 86% and 91% of the respective roofs (Fig. 3). The implementations are therefore close to optimal for this hardware; the memory subsystem, not the arithmetic units or the code, limits performance — precisely the behaviour HPCG is designed to expose [1], [10], and consistent with SpMV studies on multicore platforms [6].

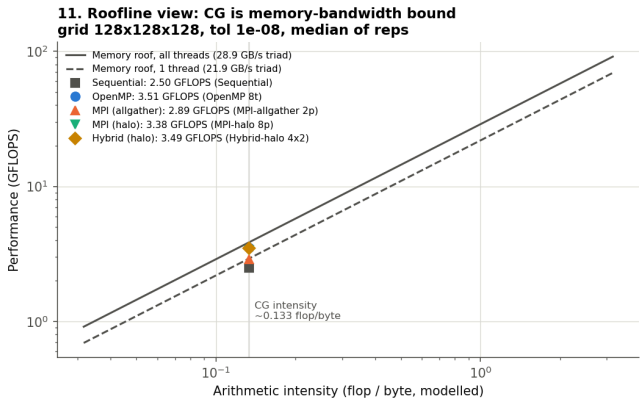


FIG. 3. Roofline with measured triad bandwidth roofs. All implementations lie just under the bandwidth bound at I ≈ 0.133 flop/byte.

E. Problem size and convergence

Fig. 5(a) shows that cache-resident problems ($\leq 64^3$) yield OpenMP speedups of 2.7–3.9× and apparent bandwidths above the DRAM roof; beyond 96^3 data streams from DRAM and the speedup collapses to ≈1.7×. Small benchmark problems therefore overstate parallel performance, which motivates HPCG's minimum problem-size rules. The iteration counts 41, 81, 120, 158, 225, 296 and 368 for N = 16…160 give k/N ≈ 2.3–2.6, confirming the linear growth predicted by (2)–(3) (Fig. 4).

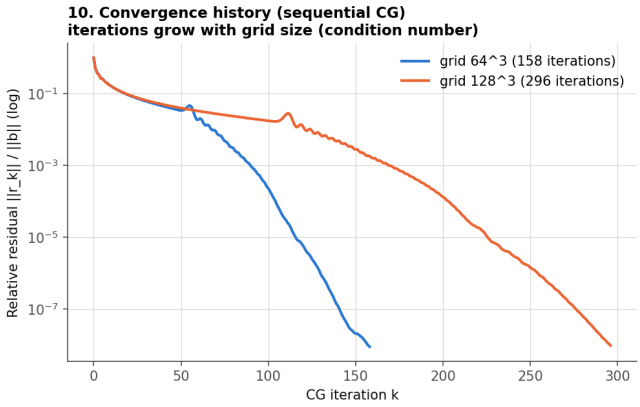


FIG. 4. Relative residual history of sequential CG for 64^3 and 128^3 grids; larger grids (higher κ) need proportionally more iterations.

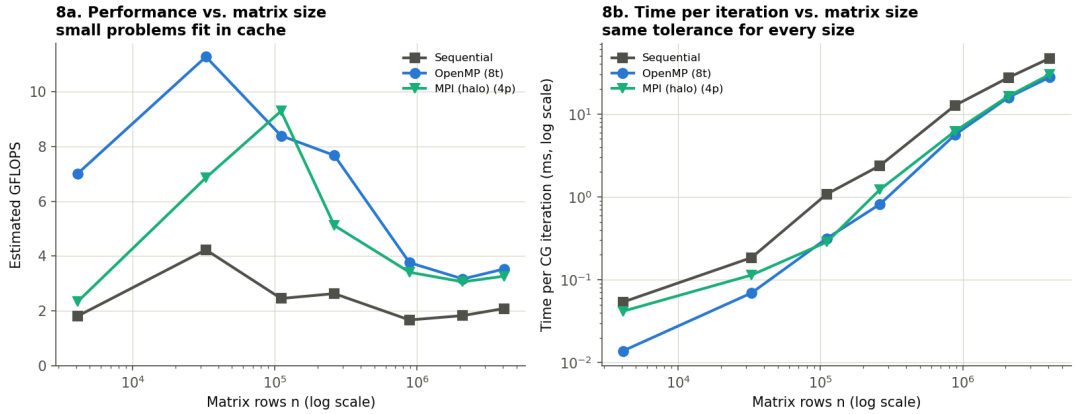


FIG. 5. (a) Estimated GFLOPS and (b) time per iteration versus matrix size for sequential, OpenMP (8 threads) and MPI-halo (4 processes), same tolerance for every size.

VI. Discussion and Limitations

The experiments separate two effects. *Shared bandwidth* limits all models equally on one node; on a cluster each node adds its own memory channels, so distributed scaling would improve and the latency of global `MPI_Allreduce` would become the bottleneck. *Communication design* matters independently: replicated vectors waste bandwidth that halo exchange saves. Limitations: (i) a single laptop, so MPI uses shared memory rather than a network; (ii) no multigrid/Gauss–Seidel preconditioner, a 7-point instead of 27-point stencil and a 1-D decomposition, so the results are not comparable with official HPCG; (iii) virtualisation and thermal effects add variance; (iv) memory traffic is modelled because hardware counters were unavailable in the container.

VII. Conclusion

We implemented and verified sequential, OpenMP, MPI and hybrid CG solvers for a 2.1-million-unknown sparse SPD system. All converge identically, and their performance is explained quantitatively: speedup saturates at 1.40× because memory bandwidth grows only 1.32×, all versions reach ≈90% of the roofline bound, and halo exchange outperforms replicated-vector communication by 1.69×. For sparse scientific workloads, bytes per second matter more than flops per second — the core rationale for HPCG as a complement to HPL. Future work: multigrid preconditioning, 3-D decomposition, communication–computation overlap, and multi-node and GPU evaluation.

References

[1] J. Dongarra and M. A. Heroux, "Toward a new metric for ranking high performance computing systems," Sandia Nat. Lab., Tech. Rep. SAND2013-4744, 2013.

[2] J. Dongarra, M. A. Heroux, and P. Luszczek, "High-performance conjugate-gradient benchmark: A new metric for ranking high-performance computing systems," *Int. J. High Perform. Comput. Appl.*, vol. 30, no. 1, pp. 3–10, 2016.

[3] J. Dongarra, M. A. Heroux, and P. Luszczek, "A new metric for ranking high-performance computing systems," *Nat. Sci. Rev.*, vol. 3, no. 1, pp. 30–35, 2016.

[4] M. R. Hestenes and E. Stiefel, "Methods of conjugate gradients for solving linear systems," *J. Res. Nat. Bur. Stand.*, vol. 49, no. 6, pp. 409–436, 1952.

[5] Y. Saad, *Iterative Methods for Sparse Linear Systems*, 2nd ed. Philadelphia, PA, USA: SIAM, 2003.

[6] S. Williams, L. Oliker, R. Vuduc, J. Shalf, K. Yelick, and J. Demmel, "Optimization of sparse matrix–vector multiplication on emerging multicore platforms," *Parallel Comput.*, vol. 35, no. 3, pp. 178–194, 2009.

[7] J. J. Dongarra, P. Luszczek, and A. Petitet, "The LINPACK benchmark: Past, present and future," *Concurrency Comput.: Pract. Exper.*, vol. 15, no. 9, pp. 803–820, 2003.

[8] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Commun. ACM*, vol. 52, no. 4, pp. 65–76, 2009.

[9] J. D. McCalpin, "Memory bandwidth and machine balance in current high performance computers," *IEEE TCCA Newsletter*, Dec. 1995.

[10] V. Marjanović, J. Gracia, and C. W. Glass, "Performance modeling of the HPCG benchmark," in *Proc. PMBS 2014*, LNCS 8966, Springer, 2015, pp. 172–192.

[11] R. Rabenseifner, G. Hager, and G. Jost, "Hybrid MPI/OpenMP parallel programming on clusters of multi-core SMP nodes," in *Proc. 17th Euromicro PDP*, 2009, pp. 427–436.

Code, raw data (results/benchmark.csv, 50 configurations) and run logs: github.com/Anuttaman216/HPC_FInal_Lab_Project. HPCG-inspired educational implementation; not the official HPCG benchmark.